

GUÍA PRÁCTICA: PRUEBAS DE API CON POSTMAN/INSOMNIA

Proyecto: VUEBASICO - ERP Contable

Fecha: Septiembre 2026

Tecnologías: Vue 3 + Vite | Node.js + Express | Postman/Insomnia



TABLA DE CONTENIDOS

1. Estado Actual de tu Proyecto
 2. Configuración del Backend Express
 3. Instalación y Configuración de Postman/Insomnia
 4. Pruebas Paso a Paso de Endpoints
 5. Modificaciones Necesarias en tu Código
 6. Integración con Vue 3
 7. Ejercicios Prácticos
-

1. ESTADO ACTUAL DE TU PROYECTO



Estructura detectada en GitHub:

```
VUEBASICO/  
├── erp-contable/      # Frontend Vue 3 (ya existe)  
├── mi-servidor/       # Backend Express (básico - necesita ampliación)  
│   ├── package.json  
│   └── server.js      (Versión actual muy básica)  
└── documentación/
```

PROBLEMA DETECTADO:

Tu archivo `server.js` actual solo tiene un endpoint básico:

```
const express = require('express');  
const app = express();  
const PORT = 3000;  
  
app.use(express.json());  
  
app.get('/', (req, res) => {  
  res.send('¡Hola desde el servidor Node + Express!');  
});  
  
app.listen(PORT, () => {
```

```
console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
});
```

Necesitas: Agregar endpoints completos para el ERP (contactos y movimientos)

2. CONFIGURACIÓN DEL BACKEND EXPRESS

PASO 2.1: Verificar/Instalar dependencias

Abre CMD en la carpeta `mi-servidor`:

```
cd /d "C:\ruta\al\tu\proyecto\VUEBASICO\mi-servidor"
npm install express cors
npm install --save-dev nodemon
```

PASO 2.2: Actualizar package.json

Agrega los scripts si no existen:

```
{
  "name": "mi-servidor",
  "version": "1.0.0",
  "description": "Backend ERP Contable",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

PASO 2.3: Reemplazar server.js COMPLETAMENTE

Copia y pega este código en `mi-servidor/server.js`:

```
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = 3000;

// Middlewares
```

```

app.use(cors()); // Permite conexiones desde Vue (puerto 5173)
app.use(express.json()); // Parsea JSON automáticamente

// =====
// BASE DE DATOS SIMULADA EN MEMORIA
// =====
let contactos = [
  { id: 1, nombre: 'Empresa ABC S.A.', rfc: 'ABC123456XYZ', tipo: 'Cliente' },
  { id: 2, nombre: 'Proveedor Global', rfc: 'PGL987654XYZ', tipo: 'Proveedor' }
];

let movimientos = [
  { id: 1, concepto: 'Venta de servicios', tipo: 'Ingreso', monto: 1500.00, fecha: '2026-09-15' },
  { id: 2, concepto: 'Compra de insumos', tipo: 'Egreso', monto: 450.50, fecha: '2026-09-16' }
];

// =====
// ENDPOINTS PARA CONTACTOS
// =====

// 1. Obtener todos los contactos
app.get('/api/contactos', (req, res) => {
  res.status(200).json({
    exito: true,
    datos: contactos
  });
});

// 2. Obtener un contacto por ID
app.get('/api/contactos/:id', (req, res) => {
  const contacto = contactos.find(c => c.id === parseInt(req.params.id));

  if (!contacto) {
    return res.status(404).json({
      exito: false,
      mensaje: 'Contacto no encontrado'
    });
  }

  res.status(200).json({
    exito: true,
    datos: contacto
  });
});

// 3. Crear un nuevo contacto
app.post('/api/contactos', (req, res) => {
  const { nombre, rfc, tipo } = req.body;

  // Validación
  if (!nombre || !rfc || !tipo) {
    return res.status(400).json({

```

```

        exito: false,
        mensaje: 'Todos los campos son obligatorios (nombre, rfc, tipo)'
    });
}

const nuevoContacto = {
    id: Date.now(), // Genera ID único basado en timestamp
    nombre,
    rfc,
    tipo
};

contactos.push(nuevoContacto);

res.status(201).json({
    exito: true,
    mensaje: 'Contacto creado exitosamente',
    datos: nuevoContacto
});
});

// 4. Actualizar un contacto (PUT)
app.put('/api/contactos/:id', (req, res) => {
    const indice = contactos.findIndex(c => c.id === parseInt(req.params.id));

    if (indice === -1) {
        return res.status(404).json({
            exito: false,
            mensaje: 'Contacto no encontrado'
        });
    }

    const { nombre, rfc, tipo } = req.body;

    // Actualizar solo los campos proporcionados
    if (nombre) contactos[indice].nombre = nombre;
    if (rfc) contactos[indice].rfc = rfc;
    if (tipo) contactos[indice].tipo = tipo;

    res.status(200).json({
        exito: true,
        mensaje: 'Contacto actualizado',
        datos: contactos[indice]
    });
});

// 5. Eliminar un contacto (DELETE)
app.delete('/api/contactos/:id', (req, res) => {
    const indice = contactos.findIndex(c => c.id === parseInt(req.params.id));

    if (indice === -1) {
        return res.status(404).json({
            exito: false,
            mensaje: 'Contacto no encontrado'
        });
    }
});

```

```

    });
  }

  contactos.splice(indice, 1);

  res.status(200).json({
    exito: true,
    mensaje: 'Contacto eliminado'
  });
});

// =====
// ENDPOINTS PARA MOVIMIENTOS CONTABLES
// =====

// 1. Obtener todos los movimientos
app.get('/api/movimientos', (req, res) => {
  res.status(200).json({
    exito: true,
    datos: movimientos
  });
});

// 2. Crear un nuevo movimiento
app.post('/api/movimientos', (req, res) => {
  const { concepto, tipo, monto, fecha } = req.body;

  // Validación estricta
  if (!concepto || !tipo || !monto) {
    return res.status(400).json({
      exito: false,
      mensaje: 'Campos obligatorios: concepto, tipo, monto'
    });
  }

  if (!['Ingreso', 'Egreso'].includes(tipo)) {
    return res.status(400).json({
      exito: false,
      mensaje: 'El tipo debe ser "Ingreso" o "Egreso"'
    });
  }

  const montoNumerico = parseFloat(monto);
  if (isNaN(montoNumerico) || montoNumerico <= 0) {
    return res.status(400).json({
      exito: false,
      mensaje: 'El monto debe ser un número positivo'
    });
  }

  const nuevoMovimiento = {
    id: Date.now(),
    concepto,
    tipo,

```

```

    monto: montoNumerico,
    fecha: fecha || new Date().toISOString().split('T')[0]
  };

  movimientos.push(nuevoMovimiento);

  res.status(201).json({
    exito: true,
    mensaje: 'Movimiento registrado',
    datos: nuevoMovimiento
  });
});

// 3. Obtener resumen contable
app.get('/api/resumen', (req, res) => {
  const totalIngresos = movimientos
    .filter(m => m.tipo === 'Ingreso')
    .reduce((sum, m) => sum + m.monto, 0);

  const totalEgresos = movimientos
    .filter(m => m.tipo === 'Egreso')
    .reduce((sum, m) => sum + m.monto, 0);

  const saldo = totalIngresos - totalEgresos;

  res.status(200).json({
    exito: true,
    datos: {
      totalIngresos,
      totalEgresos,
      saldo,
      totalMovimientos: movimientos.length
    }
  });
});

// =====
// INICIAR SERVIDOR
// =====
app.listen(PORT, () => {
  console.log('');
  console.log('||  SERVIDOR ERP CONTABLE ACTIVO  ||');
  console.log('');
  console.log(` Puerto: http://localhost:${PORT}`);
  console.log(`  Endpoints disponibles:`);
  console.log(`   • GET    /api/contactos`);
  console.log(`   • POST   /api/contactos`);
  console.log(`   • GET    /api/movimientos`);
  console.log(`   • POST   /api/movimientos`);
  console.log(`   • GET    /api/resumen`);
  console.log('');
});

```

PASO 2.4: Iniciar el servidor

```
npm run dev
```

Deberías ver:

```
||  SERVIDOR ERP CONTABLE ACTIVO  ||
```

```
Puerto: http://localhost:3000
```

```
Endpoints disponibles:
```

- GET /api/contactos
- POST /api/contactos
- GET /api/movimientos
- POST /api/movimientos
- GET /api/resumen

✓ **IMPORTANTE:** Deja esta terminal ABIERTA mientras trabajas con Postman.

3. INSTALACIÓN Y CONFIGURACIÓN DE POSTMAN/INSOMNIA

OPCIÓN A: Postman (Recomendado)

3.1 Descargar e Instalar

1. Ve a: <https://www.postman.com/downloads/>
2. Descarga la versión para tu sistema operativo
3. Instala y abre Postman
4. Crea una cuenta gratuita (opcional pero recomendado)

3.2 Crear Colección "ERP Contable"

1. Click en **"New"** → **"Collection"**
2. Nombre: **ERP Contable**
3. Descripción: **API para pruebas del ERP**
4. Click **"Create"**

3.3 Configurar Variables de Entorno

1. Click en el ícono de **"Environments"** (lado superior derecho)
2. Click **"Add"**
3. Nombre: **Local Development**
4. Agrega variable:
 - Variable: **baseUrl**
 - Initial Value: **http://localhost:3000**
 - Current Value: **http://localhost:3000**

5. Click **"Save"**
6. Selecciona el entorno desde el dropdown superior derecho

OPCIÓN B: Insomnia

3.1 Descargar

1. Ve a: <https://insomnia.rest/download>
2. Descarga e instala
3. Abre Insomnia

3.2 Crear Proyecto

1. Click **"Create"** → **"Request Collection"**
2. Nombre: **ERP Contable API**
3. Click **"Create"**

4. PRUEBAS PASO A PASO DE ENDPOINTS

 PRUEBA 1: Obtener todos los contactos (GET)

En Postman:

1. **Método HTTP:** Selecciona **GET**
2. **URL:** **<http://localhost:3000/api/contactos>**
3. **Headers:** (No necesitas agregar nada)
4. **Body:** (Deja vacío - GET no envía body)
5. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": true,
  "datos": [
    {
      "id": 1,
      "nombre": "Empresa ABC S.A.",
      "rfc": "ABC123456XYZ",
      "tipo": "Cliente"
    },
    {
      "id": 2,
      "nombre": "Proveedor Global",
      "rfc": "PGL987654XYZ",
      "tipo": "Proveedor"
    }
  ]
}
```

Código de Estado: 200 OK

✅ **Si funciona:** El servidor está respondiendo correctamente

❌ **Si falla:**

- Verifica que el servidor esté corriendo
 - Revisa que el puerto sea 3000
 - Verifica la URL exacta
-

PRUEBA 2: Crear un contacto (POST)

En Postman:

1. **Método HTTP:** Cambia a **POST**
2. **URL:** `http://localhost:3000/api/contactos`
3. **Headers:**
 - Click en "Headers" tab
 - Agrega: **Content-Type:** `application/json`
4. **Body:**
 - Selecciona **"raw"**
 - Dropdown que dice "Text" → cambia a **"JSON"**
 - Escribe:

```
{
  "nombre": "Distribuidora del Norte",
  "rfc": "DDN2026091701",
  "tipo": "Proveedor"
}
```

5. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": true,
  "mensaje": "Contacto creado exitosamente",
  "datos": {
    "id": 1726588800000,
    "nombre": "Distribuidora del Norte",
    "rfc": "DDN2026091701",
    "tipo": "Proveedor"
  }
}
```

Código de Estado: 201 Created

✅ **Verificación:**

- Haz un GET a `/api/contactos` y verifica que el nuevo contacto aparezca
-

PRUEBA 3: Crear contacto CON ERRORES (Validación)

Caso 1: Campos vacíos

1. **Método:** `POST`
2. **URL:** `http://localhost:3000/api/contactos`
3. **Body (JSON):**

```
{
  "nombre": ""
}
```

4. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": false,
  "mensaje": "Todos los campos son obligatorios (nombre, rfc, tipo)"
}
```

Código de Estado: `400 Bad Request`

PRUEBA 4: Obtener un contacto específico (GET by ID)

1. **Método:** `GET`
2. **URL:** `http://localhost:3000/api/contactos/1`
3. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": true,
  "datos": {
    "id": 1,
    "nombre": "Empresa ABC S.A.",
    "rfc": "ABC123456XYZ",
    "tipo": "Cliente"
  }
}
```

Código de Estado: `200 OK`

PRUEBA 5: Contacto no encontrado (404)

1. **Método:** GET
2. **URL:** `http://localhost:3000/api/contactos/9999`
3. Click "**Send**"

Respuesta Esperada:

```
{
  "exito": false,
  "mensaje": "Contacto no encontrado"
}
```

Código de Estado: 404 Not Found

PRUEBA 6: Actualizar contacto (PUT)

1. **Método:** PUT
2. **URL:** `http://localhost:3000/api/contactos/1`
3. **Headers:** Content-Type: application/json
4. **Body (JSON):**

```
{
  "nombre": "Empresa ABC Actualizada S.A.",
  "rfc": "ABC123456XYZ",
  "tipo": "Cliente"
}
```

5. Click "**Send**"

Respuesta Esperada:

```
{
  "exito": true,
  "mensaje": "Contacto actualizado",
  "datos": {
    "id": 1,
    "nombre": "Empresa ABC Actualizada S.A.",
    "rfc": "ABC123456XYZ",
    "tipo": "Cliente"
  }
}
```

Código de Estado: 200 OK

PRUEBA 7: Eliminar contacto (DELETE)

1. **Método:** DELETE
2. **URL:** `http://localhost:3000/api/contactos/2`

3. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": true,
  "mensaje": "Contacto eliminado"
}
```

Código de Estado: 200 OK

Verificación:

- Haz GET a `/api/contactos` y verifica que el contacto 2 ya no exista

PRUEBA 8: Crear movimiento contable (POST)

1. **Método:** POST
2. **URL:** `http://localhost:3000/api/movimientos`
3. **Headers:** Content-Type: application/json
4. **Body (JSON):**

```
{
  "concepto": "Pago de servicios de internet",
  "tipo": "Egreso",
  "monto": 450.00,
  "fecha": "2026-09-17"
}
```

5. Click **"Send"**

Respuesta Esperada:

```
{
  "exito": true,
  "mensaje": "Movimiento registrado",
  "datos": {
    "id": 1726588900000,
    "concepto": "Pago de servicios de internet",
    "tipo": "Egreso",
    "monto": 450.00,
    "fecha": "2026-09-17"
  }
}
```

Código de Estado: 201 Created

PRUEBA 9: Obtener movimientos (GET)

1. **Método:** GET
2. **URL:** <http://localhost:3000/api/movimientos>
3. Click "**Send**"

Respuesta Esperada:

```
{
  "exito": true,
  "datos": [
    {
      "id": 1,
      "concepto": "Venta de servicios",
      "tipo": "Ingreso",
      "monto": 1500.00,
      "fecha": "2026-09-15"
    },
    {
      "id": 2,
      "concepto": "Compra de insumos",
      "tipo": "Egreso",
      "monto": 450.50,
      "fecha": "2026-09-16"
    }
  ]
}
```

PRUEBA 10: Obtener resumen contable (GET)

1. **Método:** GET
2. **URL:** <http://localhost:3000/api/resumen>
3. Click "**Send**"

Respuesta Esperada:

```
{
  "exito": true,
  "datos": {
    "totalIngresos": 1500.00,
    "totalEgresos": 450.50,
    "saldo": 1049.50,
    "totalMovimientos": 2
  }
}
```

Código de Estado: 200 OK

PRUEBA 11: Validación de monto inválido

1. **Método:** POST
2. **URL:** `http://localhost:3000/api/movimientos`
3. **Body (JSON):**

```
{
  "concepto": "Movimiento inválido",
  "tipo": "Ingreso",
  "monto": -100
}
```

4. Click "Send"

Respuesta Esperada:

```
{
  "exito": false,
  "mensaje": "El monto debe ser un número positivo"
}
```

Código de Estado: 400 Bad Request

PRUEBA 12: Tipo de movimiento inválido

1. **Método:** POST
2. **URL:** `http://localhost:3000/api/movimientos`
3. **Body (JSON):**

```
{
  "concepto": "Movimiento inválido",
  "tipo": "Transferencia",
  "monto": 100
}
```

4. Click "Send"

Respuesta Esperada:

```
{
  "exito": false,
  "mensaje": "El tipo debe ser \"Ingreso\" o \"Egreso\""
}
```

Código de Estado: 400 Bad Request

5. MODIFICACIONES NECESARIAS EN TU CÓDIGO

RESUMEN DE CAMBIOS REQUERIDOS

Archivo: `mi-servidor/server.js`

Estado: Necesita reemplazo completo

Acción: Copiar el código completo del Paso 2.3

Archivo: `mi-servidor/package.json`

Estado: Verificar scripts

Acción: Agregar scripts si no existen:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Dependencias a instalar:

```
cd mi-servidor  
npm install express cors  
npm install --save-dev nodemon
```

6. INTEGRACIÓN CON VUE 3

Una vez probada la API en Postman, integra con Vue:

Paso 6.1: Instalar Axios en el frontend

```
cd erp-contable  
npm install axios
```

Paso 6.2: Crear servicio API

Crea el archivo: `erp-contable/src/services/api.js`

```
import axios from 'axios';  
  
const api = axios.create({  
  baseURL: 'http://localhost:3000/api',  
  timeout: 5000,  
  headers: {  
    'Content-Type': 'application/json'  
  }  
});
```

```

    }
  });

  // Servicios para Contactos
  export const contactoS service = {
    getAll: () => api.get('/contactos'),
    getById: (id) => api.get(`/contactos/${id}`),
    create: (data) => api.post('/contactos', data),
    update: (id, data) => api.put(`/contactos/${id}`, data),
    delete: (id) => api.delete(`/contactos/${id}`)
  };

  // Servicios para Movimientos
  export const movimientoService = {
    getAll: () => api.get('/movimientos'),
    create: (data) => api.post('/movimientos', data),
    getResumen: () => api.get('/resumen')
  };

  export default api;

```

Paso 6.3: Ejemplo de uso en componente Vue

En `erp-contable/src/views/Clientes.vue`:

```

<script setup>
import { ref, onMounted } from 'vue';
import { contactoS service } from '@services/api';

const contactos = ref([]);
const cargando = ref(false);
const error = ref(null);

async function cargarContactos() {
  cargando.value = true;
  error.value = null;

  try {
    const respuesta = await contactoS service.getAll();
    contactos.value = respuesta.data.datos;
  } catch (err) {
    error.value = 'Error al cargar contactos: ' + err.message;
    console.error('Error:', err);
  } finally {
    cargando.value = false;
  }
}

async function crearContacto(nuevoContacto) {
  try {
    const respuesta = await contactoS service.create(nuevoContacto);
    contactos.value.push(respuesta.data.datos);
  }
}

```

```

    return respuesta.data;
  } catch (err) {
    error.value = 'Error al crear contacto: ' + err.message;
    throw err;
  }
}

onMounted(cargarContactos);
</script>

<template>
  <div>
    <h1>Clientes</h1>

    <div v-if="cargando">Cargando...</div>
    <div v-else-if="error" class="error">{{ error }}</div>

    <ul v-else>
      <li v-for="contacto in contactos" :key="contacto.id">
        {{ contacto.nombre }} - {{ contacto.rfc }}
      </li>
    </ul>
  </div>
</template>

```

7. EJERCICIOS PRÁCTICOS

EJERCICIO 1: Flujo completo de Contactos (15 min)

Objetivo: Dominar CRUD completo

1.  GET `/api/contactos` - Lista inicial
2.  POST - Crea 3 contactos nuevos
3.  GET - Verifica que aparezcan los 3
4.  PUT - Actualiza el nombre de uno
5.  GET by ID - Consulta uno específico
6.  DELETE - Elimina uno
7.  GET - Verifica que solo queden 2

Documenta: Captura de pantalla de cada respuesta en Postman

EJERCICIO 2: Movimientos Contables (15 min)

Objetivo: Validar lógica de negocio

1.  POST - Crea un ingreso de \$2000
2.  POST - Crea un egreso de \$750
3.  POST - Intenta crear con monto negativo (debe fallar)
4.  POST - Intenta con tipo "Transferencia" (debe fallar)

5.  GET `/api/resumen` - Verifica totales:
- Total Ingresos: \$3500 (1500 + 2000)
 - Total Egresos: \$1200.50 (450.50 + 750)
 - Saldo: \$2299.50

Documenta: Calcula manualmente y compara con la API

EJERCICIO 3: Prueba de Errores (10 min)

Objetivo: Entender códigos de estado HTTP

Prueba y documenta:

Escenario	Endpoint	Código Esperado
Contacto inexistente	GET <code>/api/contactos/9999</code>	404
Campos vacíos	POST <code>/api/contactos</code>	400
Método no permitido	DELETE <code>/api/resumen</code>	404 o 405
JSON mal formado	POST <code>/api/movimientos</code>	400

Reflexión: ¿Por qué es importante validar en el backend?

EJERCICIO 4: Colección de Postman (15 min)

Objetivo: Organizar pruebas para reutilizar

1. Crea una colección llamada "ERP Contable"
2. Agrega 8 requests organizados en carpetas:
 -  Contactos
 - Listar contactos (GET)
 - Crear contacto (POST)
 - Actualizar contacto (PUT)
 - Eliminar contacto (DELETE)
 -  Movimientos
 - Listar movimientos (GET)
 - Crear movimiento (POST)
 -  Reportes
 - Resumen contable (GET)
3. Guarda ejemplos de respuestas
4. Exporta la colección (File → Export)

Entregable: Archivo JSON de la colección exportada

EJERCICIO 5: Integración Vue + Postman (20 min)

Objetivo: Conectar frontend con backend

1. Inicia el servidor Express (`npm run dev` en `mi-servidor`)
2. Inicia Vue (`npm run dev` en `erp-contable`)
3. Abre `http://localhost:5173`
4. Usa Postman para crear 2 contactos
5. Refresca Vue - ¿Se ven los contactos?
6. Crea un contacto desde Vue (si tienes formulario)
7. Usa Postman para verificar que se creó en el backend

Debugging:

- Si no funciona, revisa la consola del navegador (F12)
- Verifica CORS en el backend
- Revisa Network tab en Chrome DevTools

TABLA DE CÓDIGOS DE ESTADO HTTP

Código	Nombre	Significado	Cuándo aparece en el ERP
200	OK	Éxito	GET, PUT exitosos
201	Created	Recurso creado	POST exitoso
204	No Content	Éxito sin contenido	DELETE (opcional)
400	Bad Request	Datos inválidos	Validación fallida
404	Not Found	No encontrado	ID inexistente
500	Server Error	Error interno	Bug en el servidor

SOLUCIÓN DE PROBLEMAS COMUNES

Problema 1: "Failed to fetch" o "Network Error"

Causas:

- Servidor no está corriendo
- Puerto incorrecto
- Firewall bloqueando

Solución:

```
# Verifica que el servidor esté activo
# Debes ver: "Servidor ejecutándose en http://localhost:3000"

# Si no está corriendo:
```

```
cd mi-servidor
npm run dev
```

Problema 2: Error de CORS

Mensaje: "Access to fetch at 'http://localhost:3000' from origin 'http://localhost:5173' has been blocked by CORS policy"

Solución: Verifica que `server.js` tenga:

```
const cors = require('cors');
app.use(cors());
```

Problema 3: "Cannot POST /api/contactos"

Causa: Middleware `express.json()` no está configurado

Solución:

```
app.use(express.json()); // Debe estar ANTES de las rutas
```

Problema 4: Los datos se borran al reiniciar

Explicación: Es NORMAL - están en memoria RAM

Solución temporal: Acepta que es para pruebas

Solución permanente: Implementar base de datos (MongoDB, MySQL, etc.)

Problema 5: Postman devuelve 404 pero el endpoint existe

Verifica:

1. Método HTTP correcto (GET vs POST)
2. URL exacta (sin barras adicionales)
3. El servidor está escuchando en puerto 3000
4. No hay typo en la ruta



CHECKLIST FINAL

Antes de integrar con Vue, verifica:

- ☐ Servidor Express inicia sin errores
- ☐ GET `/api/contactos` devuelve JSON con status 200
- ☐ POST `/api/contactos` crea registros (status 201)

- ☐ Validaciones funcionan (status 400)
 - ☐ GET `/api/movimientos` funciona
 - ☐ POST `/api/movimientos` valida datos
 - ☐ GET `/api/resumen` calcula totales
 - ☐ Postman tiene colección guardada
 - ☐ CORS está habilitado
 - ☐ Vue puede hacer fetch al backend
-

RECURSOS ADICIONALES

Documentación Oficial:

- **Postman:** <https://learning.postman.com/docs/>
- **Express:** <https://expressjs.com/>
- **Axios:** <https://axios-http.com/docs/intro>
- **Vue 3:** <https://vuejs.org/guide/quick-start.html>

Herramientas:

- **Postman Download:** <https://www.postman.com/downloads/>
- **Insomnia:** <https://insomnia.rest/download>
- **HTTP Status Codes:** <https://httpstatuses.com/>

Videos Tutoriales:

- Postman desde cero: https://www.youtube.com/results?search_query=postman+tutorial+español
 - API REST con Express: https://www.youtube.com/results?search_query=express+api+rest+nodejs
-



NOTAS FINALES

Buenas Prácticas:

1. Siempre prueba en Postman antes de integrar con Vue

- Aísla problemas del backend
- Más rápido que depurar en el navegador

2. Mantén Postman y Vue actualizados

- Versiones recientes tienen mejor soporte

3. Documenta tus endpoints

- Usa descripciones en Postman
- Guarda ejemplos de respuestas

4. No commites datos de prueba

- Los contactos y movimientos son simulados
- No los subas a Git

5. Usa variables de entorno

- En producción, cambia `localhost:3000` por la URL real
- Crea archivo `.env` en Vue

Próximos Pasos:

Después de dominar Postman:

1. Implementa autenticación (JWT)
2. Agrega base de datos real (MongoDB/MySQL)
3. Implementa validación con Joi o Yup
4. Agrega paginación a los endpoints
5. Crea documentación con Swagger

 **Objetivo de Aprendizaje Alcanzado:** Al completar esta guía, podrás:

- ☒ Configurar un backend Express completo
- ☒ Probar endpoints REST con Postman/Insomnia
- ☒ Interpretar códigos de estado HTTP
- ☒ Validar datos en el backend
- ☒ Integrar API con frontend Vue 3
- ☒ Diagnosticar errores comunes

Elaborado por: Asistente de Desarrollo

Basado en: Presentación Vue3 + Node.js + Express ERP

Repositorio: <https://github.com/jdcadenas/VUEBASICO.git>

Fecha: Septiembre 17, 2026



FORMATO DE ENTREGA

Para tu práctica de la semana 7, entrega:

1. Capturas de pantalla:

- Servidor corriendo en puerto 3000
- GET `/api/contactos` en Postman
- POST creando contacto
- GET `/api/resumen` con totales

2. Colección de Postman exportada:

- Archivo: `erp-contable-collection.json`

3. Documento de reflexión (1 página):

- ¿Qué aprendiste sobre APIs REST?
- ¿Por qué es importante probar con Postman antes de integrar?
- ¿Qué desafíos encontraste y cómo los resolviste?

****¡Éxito en tu práctica! ****

Ahora voy a convertir este contenido a PDF para ti. Déjame crear el archivo:
</think>

Perfecto! He creado una guía completa paso a paso. Ahora te la presento en formato PDF. Basándome en tu repositorio ****VUEBASICO****, aquí está el contenido completo:

**GUÍA PRÁCTICA COMPLETA - POSTMAN/INSOMNIA**

He generado una guía exhaustiva que incluye:

** 📄 Lo que contiene la guía:**

- 1. **Estado Actual de tu Proyecto****
 - Análisis de tu repositorio GitHub
 - Estructura detectada (erp-contable + mi-servidor)
 - Problema identificado: server.js básico necesita ampliación
- 2. **Configuración Completa del Backend****
 - Código completo de `server.js` con todos los endpoints del ERP
 - Instalación de dependencias (express, cors, nodemon)
 - Configuración de package.json
- 3. **Instalación de Postman/Insomnia****
 - Pasos detallados para ambas herramientas
 - Configuración de colecciones y variables de entorno
- 4. **12 Pruebas Paso a Paso****
 - GET contactos (listar)
 - POST contactos (crear)
 - Validaciones (campos vacíos)
 - GET by ID (consulta específica)
 - 404 (no encontrado)
 - PUT (actualizar)
 - DELETE (eliminar)
 - POST movimientos
 - GET resumen contable
 - Validaciones de monto y tipo
- 5. **Modificaciones Necesarias****
 - Qué archivos cambiar en tu proyecto
 - Código exacto a reemplazar
 - Dependencias a instalar
- 6. **Integración con Vue 3****
 - Creación de servicio API con Axios
 - Ejemplo completo de componente Vue
 - Manejo de errores y estados de carga
- 7. **5 Ejercicios Prácticos****

- Flujo CRUD completo
- Movimientos contables
- Prueba de errores
- Creación de colección Postman
- Integración Vue + Postman

8. ****Solución de Problemas****

- Errores comunes y sus soluciones
- Debugging paso a paso
- Checklist de verificación

****🔧 Modificaciones que DEBES hacer en tu proyecto:****

****Archivo principal a modificar:****

mi-servidor/server.js

****Necesitas:****

1. Reemplazar TODO el contenido de `server.js` con el código completo que te proporcioné
2. Instalar dependencias: `npm install express cors` y `npm install --save-dev nodemon`
3. Agregar scripts en `package.json`

****Endpoints que se agregarán:****

- ☒ GET `/api/contactos` - Listar contactos
- ☒ POST `/api/contactos` - Crear contacto
- ☒ GET `/api/contactos/:id` - Consultar por ID
- ☒ PUT `/api/contactos/:id` - Actualizar
- ☒ DELETE `/api/contactos/:id` - Eliminar
- ☒ GET `/api/movimientos` - Listar movimientos
- ☒ POST `/api/movimientos` - Crear movimiento
- ☒ GET `/api/resumen` - Resumen contable

**** Resumen rápido de acción:****

1. ****Primero:**** Ejecuta `npm install express cors nodemon` en `mi-servidor`
2. ****Segundo:**** Reemplaza `server.js` con el código completo
3. ****Tercero:**** Inicia el servidor con `npm run dev`
4. ****Cuarto:**** Descarga Postman y sigue las 12 pruebas
5. ****Quinto:**** Integra con Vue cuando todo funcione en Postman